

Mindfully Yours — Complete Understanding Guide

22 — Complete Understanding Guide (Plain-Language Master Reference)

What this is: one single document that explains the entire project — the concept, the architecture, the knowledge base, how a conversation actually works, and exactly what has and hasn't been built — all in plain language, with diagrams. Read this if you want the whole picture in one sitting, without jumping between the other 21 documents in this folder. Every claim here is backed by a more detailed technical document elsewhere in [documnets/understanding/](#) — those are linked throughout, but you don't need to open them to follow this one.

1. What Mindfully Yours Actually Is

Picture someone opening an app and talking — by typing, by voice, or to a friendly on-screen character — about something that's bothering them. It feels like talking to a warm, patient friend.

But behind that simple conversation, the system is doing something much more careful on **every single message**: quietly figuring out how serious the situation is, and — if it ever looks genuinely dangerous — pulling in a real human being immediately, completely independent of anything the "friendly AI" part decides.

Two very different groups use the same app:

- **Most people** — everyday stress, low mood, anxiety — get self-help conversation and coping tools.
- **A smaller number** — people who need a licensed professional, or who are in real crisis — get routed to real human help.

2. The Three Rules That Shape Everything Else

Every decision in this whole project traces back to one of these three rules. When anything is unclear, these three win.

Rule 1 — Grounded, not invented. The AI is never allowed to make up mental-health advice from its own general knowledge. Every meaningful thing it says has to come from the client's own approved clinical material. The AI's only real job is to phrase that material warmly — never to invent new advice.

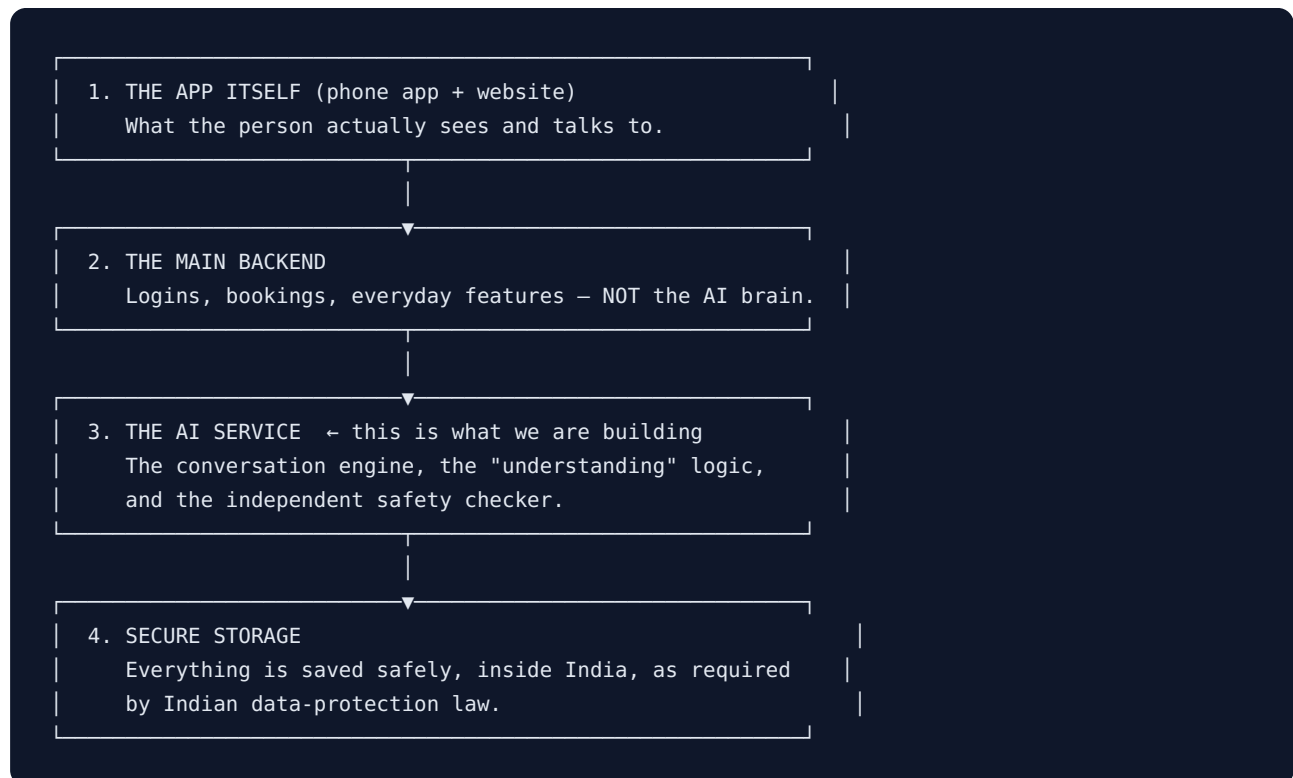
Rule 2 — Safety lives outside the AI. There is a separate, simple, rule-based checker that scans every message for danger signs. It is not "smart" — it doesn't use AI reasoning at all, on purpose. That's what makes it reliable: a fixed rule always gives the same answer, it can't be talked out of its

job, and it can't fail in a surprising way the way a "smart" system sometimes can. It runs on every message, with zero exceptions, and it can override everything else in the system.

Rule 3 — Stay human on the outside. The system should never feel like a test, a form, or a diagnostic quiz. All the "figuring out how serious this is" work happens silently, in the background, from ordinary conversation.

3. The System, Piece by Piece

Think of the whole system as four stacked layers, each with a clear, separate job:



Why is the AI Service its own separate box? So that a bug or a slow day somewhere else in the app — logins, bookings, anything — can never, even accidentally, interfere with the safety checker. This separation is deliberate architecture, not an accident of how the code happens to be organized.

Important scope note: this build (everything described in this document) covers layer 3 only — the AI Service. The app itself and the main backend are being built separately; we've documented how our layer needs to connect to them, but we are not building those parts.

4. Organizing All the Client's Material — 3 Simple Buckets

The client has sent a large amount of material over time: spreadsheets, question lists, sample conversations, self-help worksheets. Every single piece of it gets sorted into exactly one of three buckets, based on one simple question:

Is this something a person might actually say, a real thing to hand back to them, or a rule/number the system just checks?

Bucket	What goes here	How it's used
1. Teach the AI how to talk	Sample conversations, the "watch for these signs" checklist	Read once, shapes the AI's tone forever — never searched during a live chat
2. Search and either understand or recommend	Real phrases people use, question banks, self-help tools	Compared against what the user just said, either to (a) figure out what they mean, or (b) find a real exercise to offer them
3. Become fixed rules	Decision tables ("these signs mean this tier"), thresholds	Checked directly in code — no AI judgment involved at all

Concrete example of the difference: "I don't want to be here anymore" is compared (Bucket 2) against a list of known danger phrases. If it matches closely enough, the system doesn't ask the AI to write a reply — it pulls out an exact, pre-approved safety response (a Bucket 3-style rule: "if matched, use this exact text, no exceptions") and sends that instead. The AI never gets a chance to improvise here.

Full detail: [19_DATA_USAGE_MAP.md](#) and [20_IMPLEMENTATION_STRATEGIES_AND_PIPELINE.md](#).

5. How Each Bucket's "Trick" Actually Works

Bucket 1's trick — write it once, use it forever. A handful of hand-picked example conversations and the "warning signs" checklist get bundled into a fixed instruction sheet, handed to the AI at the start of every conversation. No searching, no per-message cost. It's the same idea as an actor reading a character brief once before improvising a scene.

Bucket 2's trick — turn words into a "fingerprint," then compare fingerprints. Every phrase (both the client's example phrases and, every time, whatever the user just typed) gets turned into a set of numbers that capture its meaning rather than its exact wording. That means "I don't want to be here anymore" and "I wish I could just disappear" end up with very similar fingerprints, even though the words are completely different. The system then finds the closest-matching fingerprint in its library. This is how the system understands paraphrasing and indirect language, not just exact keyword matches.

Bucket 3's trick — just check the number. No AI, no fingerprints — literally an "if this, then that" check against a table of rules, evaluated in a specific priority order the client specified. This is instant and 100% predictable, which is exactly what you want for a safety decision.

6. The Full Conversation, Step by Step

Every single message a user sends goes through this exact sequence, in this exact order, without exception:

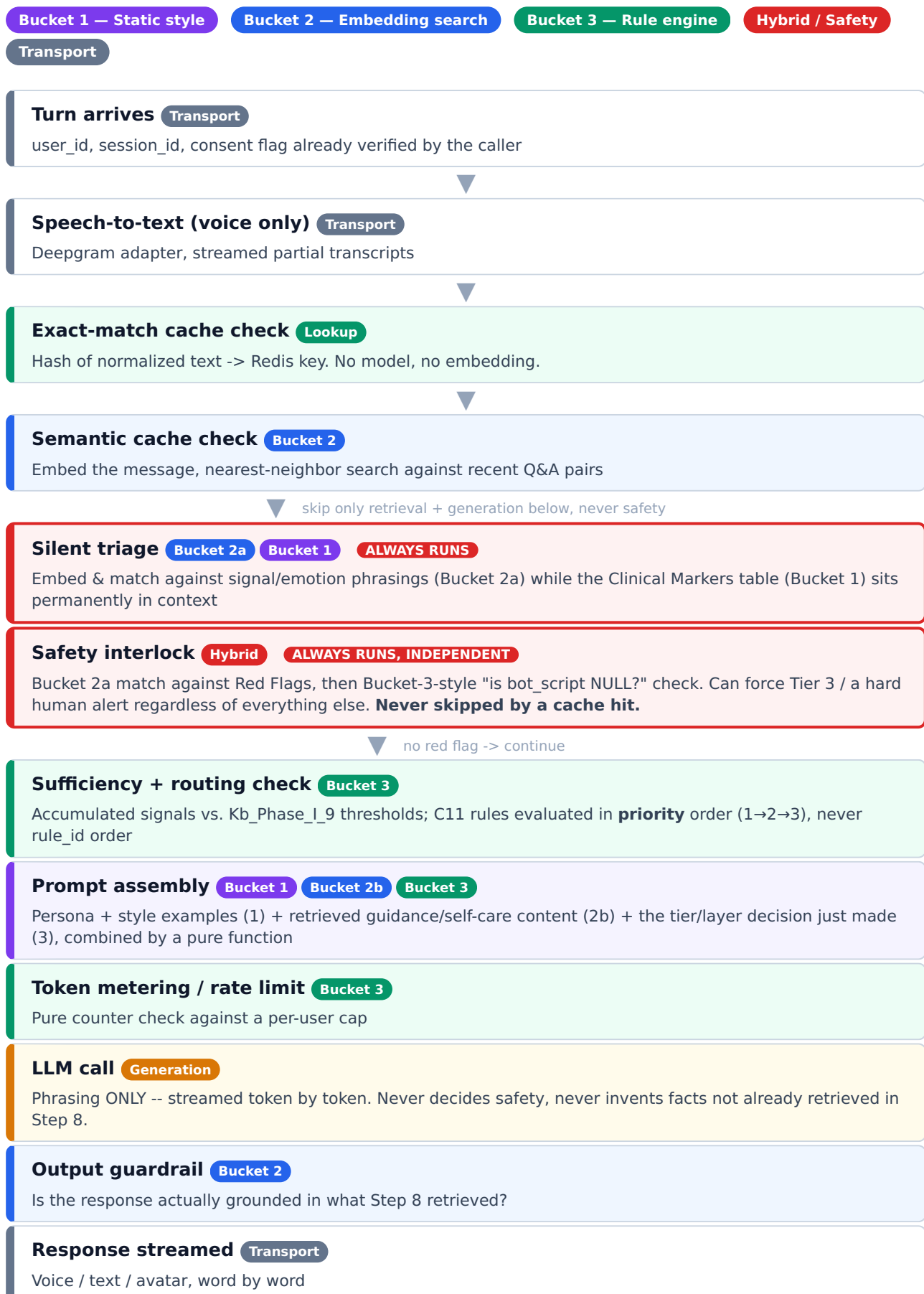
1. Message arrives (text, or voice already converted to text)
2. "Have we seen this exact message before?" → instant cached answer if yes
3. "Have we seen something very similar before?" → still-fast cached answer if yes
 - ▼ (whether or not a cache answer was found, the next two steps ALWAYS run)
4. SILENT UNDERSTANDING — quietly reads the message against the "warning signs" checklist
5. SAFETY CHECK — independently scans for danger phrases, can override everything below

```
|
▼ (if no danger signal)
6. Look up: does this match a known pattern closely enough to decide a severity level yet?
7. Build the AI's briefing note: persona + relevant real material + what's known so far
8. The AI writes ONE warm, natural reply – phrasing only, never deciding what's true or safe
9. Double-check: is the reply actually backed by real material, not just plausible-sounding?
10. Reply is sent back to the user (text, voice, or avatar), word by word as it's ready
11. In the background (never slowing the user down): save a full record, update memory for next time
```

The one rule that can never be broken: steps 4 and 5 (silent understanding and the safety check) run on **every single message**, even when a cached answer already exists in step 2 or 3. Caching is only ever allowed to skip the slow, expensive work — never the safety check.

Full detail with a color-coded visual diagram: [20_IMPLEMENTATION_STRATEGIES_AND_PIPELINE.md](#).

Figure — The Full Runtime Pipeline, Technique by Technique



Audit log + session close

Bucket 3

ASYNC, NON-BLOCKING

Full turn record written to Postgres; conversation state (Redis) updated for next turn -- fired after the response, never delaying it

The one rule that must never break: Steps 6-7 run on every turn, even when Step 3 or 4 already produced a cached answer. Caching (3-4) only ever skips retrieval and generation (5, 8, 10) -- never safety (6-7). See [01_PROJECT_OVERVIEW.md](#) principle #2.

7. How Raw Client Files Become Usable Data (Ingestion)

Before any of this can run, every spreadsheet and document the client sends has to be turned into clean, organized data the system can actually use. This happens once, offline, never while a real user is waiting:

1. Open the raw file, split it along its own natural structure (headings, rows, sections)
2. Tag each piece with a category and a language
3. Turn the real, human-phrasing pieces into "fingerprints" (only Bucket 2 content gets this)
4. Store everything in the database, properly indexed so searches stay fast
5. Run test questions through it – check two things separately: "did we find the right piece" and "was the final answer actually accurate"
6. Only if every check passes does this become the live, real version – otherwise it goes back to the clinical team

This step is allowed to take time and be careful — it never touches the fast, real-time conversation budget. Full detail: [07_INGESTION_PIPELINE.md](#) and [21_NEW_CONTENT_INGESTION_PLAN.md](#).

8. Exactly What Has Been Built — Component by Component

This is the honest, detailed status. Green means genuinely done and tested. Yellow means the shape/structure is built but the real logic inside is still a placeholder. Red means it doesn't exist yet.

Component	What it does	Status	In plain words
Understanding the whole project	All planning, architecture, and decisions written down clearly	Done	21 detailed reference documents, covering everything
Turning client files into data (Ingestion)	Reads every spreadsheet/document and turns it into organized, searchable information	Done	Built and tested against every real file the client has sent so far — both the original clinical spreadsheets and the large September document drop
Danger-phrases matching (the safety checker's "ears")	Compares a message against the list of known danger phrases	Done	Real, working code — this is the single most important piece, and it works

Component	What it does	Status	In plain words
The database design	Where everything lives — every table, every field	Done	All 19 data tables designed, built, and verified
The overall conversation sequence	The 11-step order described in Section 6	Done (structure)	The order is correctly locked in — but several steps inside it are still placeholders, see below
Two-tier caching ("have we seen this before?")	Instantly answers repeat or near-repeat questions	Half done	The "exact same question" check works; the "very similar question" check is still a stub
Silent understanding (reading a message for warning signs)	Figures out what emotional/clinical signals are present in a message	Not built yet	Currently returns nothing — this is empty scaffolding waiting to be filled in
Deciding the severity level (Tier 1 / 2 / 3)	Looks at accumulated signals and decides how serious things are	Not built yet	Currently always says "not enough information yet" — partly blocked on messy client rule data that needs a smarter reader
Writing the actual reply (the AI/LLM connection)	Calls the real AI language service to phrase a response	Not built yet	We don't have the AI vendor's real technical instructions yet, so this is a placeholder guess, not a real connection
Double-checking the reply is accurate (output guardrail)	Confirms the AI's reply is actually backed by real material	Barely started	Only the most basic possible check exists so far
Final quality testing (the validation gate)	Runs a big test question set before anything goes live	Not started — and deliberately deferred for now	Needs a test question set from the client that doesn't exist yet; agreed this is a "later" concern, not urgent

In one sentence: everything about organizing and storing the client's knowledge is done and solid. Everything about actually having a live conversation with a real AI is still mostly empty scaffolding, correctly shaped but not yet filled in.

9. The Big September Discovery — What Changed Recently

The client sent a large batch (46 files) of working documents from their clinical team. Almost the entire thing was password-locked; once unlocked and reviewed in full, several important things came out of it:

- **A whole new idea, "conversation depth" (Layer I through IV)** — separate from the existing "how serious is this" scale (Tier 1/2/3). Layer = how deep into a conversation you are (opening small talk → exploring → reflecting → action planning). Tier = how dangerous the situation is. Two different, unrelated dials.
- **A master "how the AI should think" recipe** — a clear, 15-step flowchart for how a conversation should proceed, which becomes the actual blueprint for our code.
- **Real, ready-to-use self-help tools** — a finished coping-skills workbook the AI can genuinely hand to a person who needs it. Previously we had zero of this kind of content.

- **A messy but valuable pile of extra conversation examples**, including — for the first time — real examples of how the AI should handle a genuine crisis moment (the earlier sample file had none at all).
- **Some real problems found and mostly fixed**: a "500 conversations" file that turned out to be 1 script copied 500 times (fixed — the real, useful part was pulled out and kept), a few unfinished documents, and three different "emotion word lists" from different authors that don't fully agree with each other (still needs the client's input, see below).

Full detail: [18_NEW_CONVO_DATA_CATALOG.md](#).

10. What We're Waiting On

A short list of things that can't move forward without someone else's input:

1. **Four missing "danger scripts"** — the exact wording for the most serious situations (violence, rape, etc.) simply doesn't exist yet in the client's spreadsheet.
2. **A confirmed list of emotions** — three different lists exist across the client's documents; we need one official version.
3. **Confirmation on how "Tier," "Layer," and a third "Severity" scale relate to each other** — see the plain-language question list already prepared for the client's clinical team.
4. **Real technical details from the AI voice/language vendor**, so the placeholder connection can become a real one.
5. **Three files that never unlocked** with the password we were given — likely need a different password from one specific team member.

Full list: [15_OPEN_QUESTIONS_AND_BLOCKERS.md](#).

11. Glossary — Plain-English Definitions

Term	What it means
Tier 1 / 2 / 3	How serious a situation is. 1 = everyday stress, self-help is enough. 2 = should see a real professional. 3 = urgent, alert a human right now.
Layer I-IV	How deep into a single conversation you are — separate from Tier. Opening → exploring → reflecting → planning next steps.
Red flag	A known danger phrase. If matched, the system uses an exact pre-written safety response, never an AI-generated one.
Embedding / "fingerprint"	A way of turning text into numbers that capture its meaning, so similar-meaning phrases can be found even with completely different wording.
RAG (Retrieval)	The technique of searching real approved material first, then having the AI phrase (not invent) the answer.
Ingestion	The one-time process of turning raw client files into clean, searchable data.
Bucket 1 / 2 / 3	Our own organizing system for all client material — teach tone / search & understand or recommend / fixed rules.
Pending governance review	Content that's stored but intentionally not yet allowed to be used live, because it needs a careful policy decision first.

This document is a plain-language synthesis of everything in [documnets/understanding/](#). If anything here ever seems to disagree with a more detailed document, the detailed document is the accurate one — this file should be updated to match it, not the other way around.